

Seminar 1

Grafuri Neorientate

Asist. univ. dr. Mihai Tudor

9 februarie 2026

1 Definiții fundamentale

Definiție 1.1. Un **graf neorientat** G este o pereche ordonată $G = (V, E)$, unde:

- V este o mulțime finită, nevidă, de elemente numite **vârfuri** (sau noduri);
- $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ este o mulțime de perechi neordonate de vârfuri distincte, numite **muchii**.

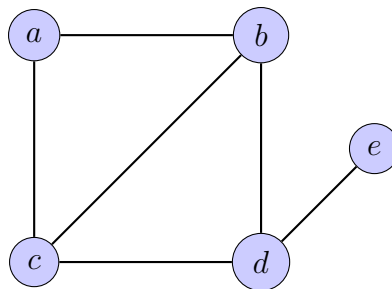
Observație 1.1. În grafurile neorientate:

- Muchia $\{u, v\}$ se mai notează și uv sau vu (ordinea nu contează);
- Nu permitem muchii multiple între aceleași două vârfuri;
- Nu permitem bucle (muchii de la un vârf la el însuși).

Exemplu 1.1. Fie graful $G_1 = (V, E)$ cu:

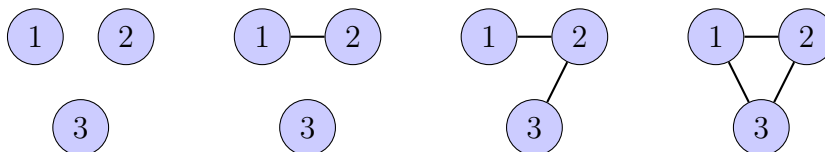
$$V = \{a, b, c, d, e\}$$

$$E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}, \{c, d\}, \{d, e\}\}$$



Exemplu 1.2. Un graf cu 3 vârfuri poate avea diferite configurații de muchii:

G_1 : 0 muchii G_2 : 1 muchie G_3 : 2 muchii G_4 : 3 muchii



Exemplu 1.3. Reprezentarea unui graf în Python folosind liste de adiacență:

```

1 # Metoda 1: Dictionar simplu
2 graf = {
3     'a': ['b', 'c'],
4     'b': ['a', 'c', 'd'],
5     'c': ['a', 'b', 'd'],
6     'd': ['b', 'c', 'e'],
7     'e': ['d']
8 }
9
10 # Afisare varfuri
11 print("Varfuri:", list(graf.keys()))

```

```

12 # Output: ['a', 'b', 'c', 'd', 'e']
13
14 # Afisare vecini pentru varful 'b'
15 print("Vecinii lui 'b':", graf['b'])
16 # Output: ['a', 'c', 'd']

```

Exemplu 1.4. Reprezentarea unui graf folosind o clasă Python:

```

1 class Graf:
2     def __init__(self):
3         """Initializeaza un graf gol"""
4         self.graf = {}
5
6     def adauga_varf(self, varf):
7         """Adauga un varf in graf"""
8         if varf not in self.graf:
9             self.graf[varf] = []
10
11     def adauga_muchie(self, u, v):
12         """Adauga o muchie intre varfurile u si v"""
13         # Adaugam varfurile daca nu exista
14         if u not in self.graf:
15             self.graf[u] = []
16         if v not in self.graf:
17             self.graf[v] = []
18
19         # Graf neorientat => adaugam in ambele directii
20         if v not in self.graf[u]:
21             self.graf[u].append(v)
22         if u not in self.graf[v]:
23             self.graf[v].append(u)
24
25     def afiseaza(self):
26         """Afiseaza graful"""
27         for varf in sorted(self.graf.keys()):
28             print(f"{varf}: {self.graf[varf]}")
29
30 # Exemplu de utilizare
31 g = Graf()
32 g.adauga_muchie('a', 'b')
33 g.adauga_muchie('a', 'c')
34 g.adauga_muchie('b', 'c')
35 g.adauga_muchie('b', 'd')
36
37 print("Graful creat:")
38 g.afiseaza()
39 # Output:
40 # a: ['b', 'c']
41 # b: ['a', 'c', 'd']
42 # c: ['a', 'b']
43 # d: ['b']

```

2 Terminologie de bază

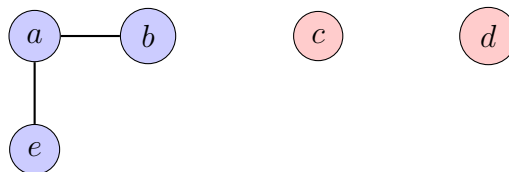
Definiție 2.1. Fie $G = (V, E)$ un graf neorientat.

- Două vârfuri $u, v \in V$ se numesc **adiacente** (sau vecine) dacă $\{u, v\} \in E$;
- O muchie $e = \{u, v\} \in E$ este **incidentă** cu vârfurile u și v ;
- **Vecinătatea** unui vârf v este mulțimea $N(v) = \{u \in V \mid \{u, v\} \in E\}$;
- **Gradul** unui vârf v este $\deg(v) = |N(v)|$, adică numărul de muchii incidente cu v .

Exemplu 2.1. Pentru graful din Exemplul 1.1:

- Vârfurile a și b sunt adiacente (există muchia $\{a, b\}$);
- Vârfurile a și d NU sunt adiacente (nu există muchie directă);
- $N(a) = \{b, c\}$;
- $N(b) = \{a, c, d\}$;
- $N(e) = \{d\}$;
- $\deg(a) = 2, \deg(b) = 3, \deg(c) = 3, \deg(d) = 3, \deg(e) = 1$.

Exemplu 2.2. Un graf cu vârfuri izolate:



Grade: $\deg(a) = 2, \deg(b) = 1, \deg(c) = 0, \deg(d) = 0, \deg(e) = 1$.

Vârfurile c și d sunt **izolate** (grad 0).

Vârfurile b și e sunt **frunze** sau **vârfuri terminale** (grad 1).

Exemplu 2.3. Calcularea gradelor vârfurilor în Python:

```

1 def calculeaza_grade(graf):
2     """Calculeaza gradul fiecarui varf"""
3     grade = {}
4     for varf in graf:
5         grade[varf] = len(graf[varf])
6     return grade
7
8 # Exemplu
9 graf = {
10     'a': ['b', 'c'],
11     'b': ['a', 'c', 'd'],
12     'c': ['a', 'b', 'd'],
13     'd': ['b', 'c', 'e'],
14     'e': ['d']
15 }
```

```

16
17 grade = calculeaza_grade(graf)
18 print("Gradele varfurilor:")
19 for varf in sorted grade.keys():
20     print(f"deg({varf}) = {grade[varf]}")
21
22 # Output:
23 # deg(a) = 2
24 # deg(b) = 3
25 # deg(c) = 3
26 # deg(d) = 3
27 # deg(e) = 1

```

Exemplu 2.4. Verificarea adiacenței:

```

1 def sunt_adiacente(graf, u, v):
2     """Verifica daca varfurile u si v sunt adiacente"""
3     if u not in graf:
4         return False
5     return v in graf[u]
6
7 graf = {'a': ['b', 'c'], 'b': ['a', 'c'], 'c': ['a', 'b']}
8
9 print(f"a si b adiacente? {sunt_adiacente(graf, 'a', 'b')}") #
10    True
11 print(f"a si c adiacente? {sunt_adiacente(graf, 'a', 'c')}") #
12    True
13 print(f"a si d adiacente? {sunt_adiacente(graf, 'a', 'd')}") #
14    False

```

Exemplu 2.5. Găsirea vârfurilor izolate:

```

1 def gaseste_varfuri_izolate(graf):
2     """Returneaza lista varfurilor izolate (grad 0)"""
3     izolate = []
4     for varf in graf:
5         if len(graf[varf]) == 0:
6             izolate.append(varf)
7     return izolate
8
9 graf = {
10     'a': ['b'],
11     'b': ['a'],
12     'c': [],
13     'd': [],
14     'e': ['f'],
15     'f': ['e']
16 }
17
18 print("Varfuri izolate:", gaseste_varfuri_izolate(graf))
19 # Output: ['c', 'd']

```

Teoremă 2.1 (Lema strângerii de mână). *Pentru orice graf $G = (V, E)$ avem:*

$$\sum_{v \in V} \deg(v) = 2|E|$$

Demonstrație. Fiecare muchie $\{u, v\} \in E$ contribuie exact cu 1 la gradul lui u și exact cu 1 la gradul lui v . Prin urmare, suma gradelor tuturor vârfurilor numără fiecare muchie de două ori. $\square$

Exemplu 2.6. Verificarea Lemei strângerii de mână pentru diverse grafuri:

Graf 1: $V = \{a, b, c, d, e\}$, $E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}, \{c, d\}, \{d, e\}\}$

Suma gradelor: $2 + 3 + 3 + 3 + 1 = 12$

Număr muchii: $|E| = 6$, deci $2|E| = 12 \checkmark$

Graf 2 (triunghi): $V = \{1, 2, 3\}$, $E = \{\{1, 2\}, \{2, 3\}, \{1, 3\}\}$

Suma gradelor: $2 + 2 + 2 = 6$

Număr muchii: $|E| = 3$, deci $2|E| = 6 \checkmark$

Exemplu 2.7. Implementare Python pentru verificarea Lemei:

```

1 def verifica_lemma_strangerii(graf):
2     """Verifica Lema strângerii de mână"""
3     # Calculează suma gradelor
4     suma_grade = sum(len(vecini) for vecini in graf.values())
5
6     # Numără muchiile (evitând dubla numărare)
7     muchii = set()
8     for u in graf:
9         for v in graf[u]:
10             muchie = tuple(sorted([u, v]))
11             muchii.add(muchie)
12
13     nr_muchii = len(muchii)
14
15     print(f"Suma gradelor: {suma_grade}")
16     print(f"Număr muchii: {nr_muchii}")
17     print(f"2      număr muchii: {2 * nr_muchii}")
18     print(f"Lema verificată: {suma_grade == 2 * nr_muchii}")
19
20     return suma_grade == 2 * nr_muchii
21
22 # Test
23 graf = {
24     'a': ['b', 'c'],
25     'b': ['a', 'c', 'd'],
26     'c': ['a', 'b', 'd'],
27     'd': ['b', 'c', 'e'],
28     'e': ['d']
29 }
30
31 verifica_lemma_strangerii(graf)

```

Propoziție 2.1. În orice graf, numărul de vârfuri cu grad impar este par.

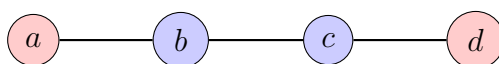
Demonstrație. Fie V_{par} mulțimea vârfurilor cu grad par și V_{impar} mulțimea vârfurilor cu grad impar. Conform Lemei strângerii de mână:

$$\sum_{v \in V_{\text{par}}} \deg(v) + \sum_{v \in V_{\text{impar}}} \deg(v) = 2|E|$$

Primul termen este par (sumă de numere pare). Membrul drept este par. Deci al doilea termen trebuie să fie par. Dar acesta este o sumă de numere impare, posibil doar dacă avem un număr par de termeni. Prin urmare, $|V_{\text{impar}}|$ este par. $\square$

Exemplu 2.8. Exemple de grafuri:

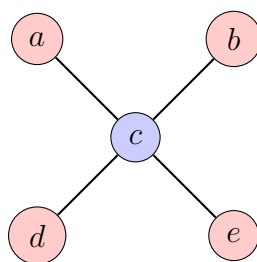
Graf 1: Lanț cu 4 vârfuri



Grade: $\deg(a) = 1$ (roșu), $\deg(b) = 2$ (albastru), $\deg(c) = 2$ (albastru), $\deg(d) = 1$ (roșu)

Vârfuri cu grad impar: 2 (par) ✓

Graf 2: Stea cu 5 vârfuri



Grade: $\deg(c) = 4$ (albastru - par), restul au grad 1 (roșu - impar)

Vârfuri cu grad impar: 4 (par) ✓

Exemplu 2.9. Verificare în Python:

```

1 def numar_varfuri_grad_impar(graf):
2     """Numara varfurile cu grad impar"""
3     count = 0
4     for varf in graf:
5         if len(graf[varf]) % 2 == 1:
6             count += 1
7     return count
8
9 # Test 1: Lanț
10 graf1 = {'a': ['b'], 'b': ['a', 'c'], 'c': ['b', 'd'], 'd': ['c']}
11 print(f"Graf lant - varfuri grad impar: {numar_varfuri_grad_impar(graf1)}")
12 # Output: 2 (a si d)
13
14 # Test 2: Triunghi
15 graf2 = {'a': ['b', 'c'], 'b': ['a', 'c'], 'c': ['a', 'b']}
```

```

16 print(f"Graf triunghi - varfuri grad impar: {
    numar_varfuri_grad_impar(graf2)}")
17 # Output: 0 (toate au grad 2)
18
19 # Test 3: Stea
20 graf3 = {'c': ['a', 'b', 'd', 'e'], 'a': ['c'], 'b': ['c'], 'd': ['c'],
    'e': ['c']}
21 print(f"Graf stea - varfuri grad impar: {numar_varfuri_grad_impar
    (graf3)}")
22 # Output: 4 (a, b, d, e)

```

3 Tipuri speciale de grafuri

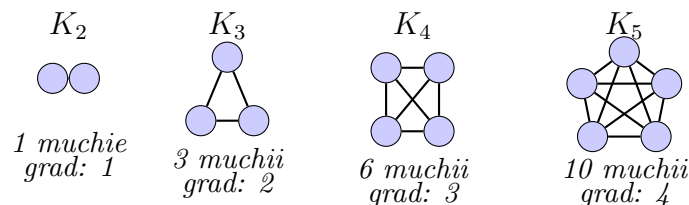
3.1 Grafuri complete

Definiție 3.1. Un graf neorientat $G = (V, E)$ se numește **graf complet** dacă oricare două vârfuri distincte sunt adiacente. Graful complet cu n vârfuri se notează K_n .

Propoziție 3.1. Pentru graful complet K_n avem:

- $|E| = \binom{n}{2} = \frac{n(n-1)}{2}$ (numărul de muchii)
- $\deg(v) = n - 1$ pentru orice $v \in V$ (fiecare vârf este conectat la toate celelalte)

Exemplu 3.1. Grafurile complete cu 2, 3, 4 și 5 vârfuri:



Verificare formule: K_5 are $\frac{5 \times 4}{2} = 10$ muchii și fiecare vârf are grad $5 - 1 = 4$.

Exemplu 3.2. Generarea unui graf complet în Python:

```

1 def genereaza_graf_complet(n):
2     """Genereaza graful complet K_n cu varfuri 1, 2, ..., n"""
3     graf = {}
4
5     # Initializam toate varfurile
6     for i in range(1, n+1):
7         graf[i] = []
8
9     # Conectam fiecare varf cu toate celelalte
10    for i in range(1, n+1):
11        for j in range(1, n+1):
12            if i != j and j not in graf[i]:
13                graf[i].append(j)
14

```



```

15     return graf
16
17 # Generam K_4
18 k4 = genereaza_graf_complet(4)
19 print("Graful K_4:")
20 for varf in sorted(k4.keys()):
21     print(f"{varf}: {k4[varf]}")
22
23 # Output:
24 # 1: [2, 3, 4]
25 # 2: [1, 3, 4]
26 # 3: [1, 2, 4]
27 # 4: [1, 2, 3]
28
29 # Verificam numarul de muchii
30 def numar_muchii(graf):
31     """Calculeaza numarul de muchii"""
32     total = sum(len(vecini) for vecini in graf.values())
33     return total // 2 # Impartim la 2 pentru graf neorientat
34
35 print(f"\nK_4 are {numar_muchii(k4)} muchii")
36 print(f"Formula: 4 * 3 / 2 = {4*3//2} muchii")

```

Exemplu 3.3. Verificare dacă un graf este complet:

```

1 def este_complet(graf):
2     """Verifica daca un graf este complet"""
3     n = len(graf)
4
5     # Toate varfurile trebuie sa aiba exact n-1 vecini
6     for varf in graf:
7         if len(graf[varf]) != n - 1:
8             return False
9
10    # Verificam ca fiecare varf e conectat cu toate celelalte
11    for varf in graf:
12        for alt_varf in graf:
13            if varf != alt_varf:
14                if alt_varf not in graf[varf]:
15                    return False
16
17    return True
18
19 # Test 1: Graf complet K_3
20 graf1 = {1: [2, 3], 2: [1, 3], 3: [1, 2]}
21 print(f"Graf1 este complet? {este_complet(graf1)}") # True
22
23 # Test 2: Graf incomplet
24 graf2 = {1: [2], 2: [1, 3], 3: [2]}
25 print(f"Graf2 este complet? {este_complet(graf2)}") # False

```

Exemplu 3.4. Tabel cu valori pentru grafuri complete:

Graf	Nr. muchii	Grad vârfuri
K_2	1	1
K_3	3	2
K_4	6	3
K_5	10	4
K_6	15	5
K_{10}	45	9
K_n	$\frac{n(n-1)}{2}$	$n - 1$

3.2 Grafuri bipartite

Definiție 3.2. Un graf $G = (V, E)$ se numește **bipartit** dacă mulțimea vârfurilor poate fi partiționată în două mulțimi V_1 și V_2 astfel încât orice muchie conectează un vârf din V_1 cu un vârf din V_2 .

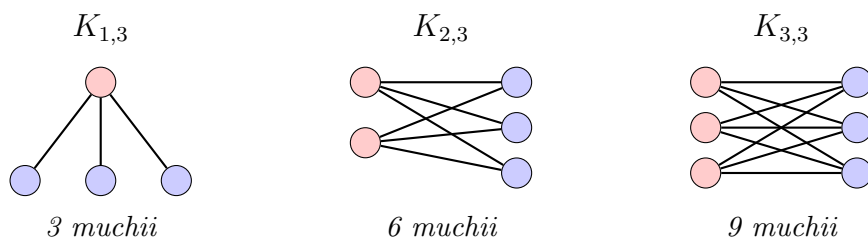
Notăție: $G = (V_1 \cup V_2, E)$ sau $G = (V_1, V_2, E)$.

Observație 3.1. Într-un graf bipartit:

- Nu există muchii între vârfuri din aceeași partiție;
- Toate muchiile conectează vârfuri din partiții diferite;
- Partițiile V_1 și V_2 pot avea dimensiuni diferite.

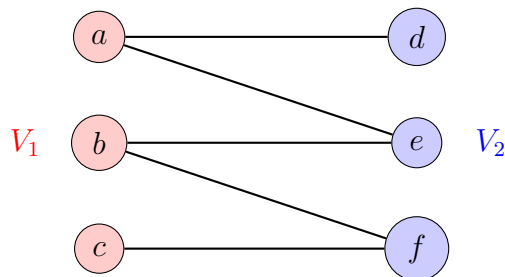
Definiție 3.3. Un graf bipartit se numește **complet bipartit** dacă orice vârf din V_1 este adiacent cu orice vârf din V_2 . Dacă $|V_1| = m$ și $|V_2| = n$, notăm acest graf cu $K_{m,n}$.

Exemplu 3.5. Exemple de grafuri bipartite complete:



Pentru $K_{m,n}$: numărul de muchii este $m \times n$.

Exemplu 3.6. Graf bipartit (nu complet):



$V_1 = \{a, b, c\}$, $V_2 = \{d, e, f\}$.

Nu este $K_{3,3}$ deoarece nu toate muchiile posibile există (de exemplu, lipsește muchia $\{a, f\}$).

Exemplu 3.7. *Exemple practice de grafuri bipartite:*

- **Rețea socială:** $V_1 = \text{utilizatori}$, $V_2 = \text{grupuri}$. Muchiile reprezintă apartenența.
- **Sistemul de recomandări:** $V_1 = \text{clienți}$, $V_2 = \text{produse}$. Muchiile = achiziții.
- **Cursuri universitare:** $V_1 = \text{studenți}$, $V_2 = \text{cursuri}$. Muchiile = înscrieri.
- **Matching job:** $V_1 = \text{candidați}$, $V_2 = \text{joburi}$. Muchiile = aplicații.

Exemplu 3.8. *Generarea unui graf bipartit complet în Python:*

```

1  def genereaza_graf_bipartit_complet(m, n):
2      """
3      Genereaza graful bipartit complet  $K_{\{m,n\}}$ 
4       $V_1 = \{a_1, a_2, \dots, a_m\}$ 
5       $V_2 = \{b_1, b_2, \dots, b_n\}$ 
6      """
7      graf = {}
8
9      # Creaza partițiile
10     v1 = [f'a{i}' for i in range(1, m+1)]
11     v2 = [f'b{j}' for j in range(1, n+1)]
12
13     # Initializeaza graful
14     for v in v1 + v2:
15         graf[v] = []
16
17     # Conecteaza fiecare varf din V1 cu fiecare din V2
18     for v_a in v1:
19         for v_b in v2:
20             graf[v_a].append(v_b)
21             graf[v_b].append(v_a)
22
23     return graf, v1, v2
24
25 # Generam  $K_{\{2,3\}}$ 
26 k23, v1, v2 = genereaza_graf_bipartit_complet(2, 3)
27 print("Graf  $K_{\{2,3\}}$ :")
28 print(f"Partitia 1: {v1}")
29 print(f"Partitia 2: {v2}")
30 print("\nListe adiacenta:")
31 for varf in sorted(k23.keys()):
32     print(f"{varf}: {k23[varf]}")
33
34 # Verificam numarul de muchii
35 print(f"\nNumar muchii: {numar_muchii(k23)}")
36 print(f"Formula: 2    3 = {2*3}")

```

Exemplu 3.9. *Verificare dacă un graf este bipartit (folosind colorare):*

```

1  def este_bipartit(graf):
2      """

```

```

3      Verifica daca un graf este bipartit folosind BFS.
4      Incercam sa coloram graful cu 2 culori (0 si 1).
5      """
6      if not graf:
7          return True, None, None
8
9      culori = {} # varf -> culoare (0 sau 1)
10
11     # Parcurgem toate componentele (pentru grafuri neconexe)
12     for start in graf:
13         if start in culori:
14             continue
15
16         # BFS din varf de start
17         coada = [start]
18         culori[start] = 0
19
20         while coada:
21             varf = coada.pop(0)
22             culoare_curenta = culori[varf]
23
24             for vecin in graf[varf]:
25                 if vecin not in culori:
26                     # Coloram vecinul cu culoarea opusa
27                     culori[vecin] = 1 - culoare_curenta
28                     coada.append(vecin)
29                 elif culori[vecin] == culoare_curenta:
30                     # Doi vecini cu aceeasi culoare => NU e
31                     bipartit
32                     return False, None, None
33
34     # Separam in cele doua partitii
35     v1 = [v for v in culori if culori[v] == 0]
36     v2 = [v for v in culori if culori[v] == 1]
37
38     return True, v1, v2
39
40 # Test 1: Graf bipartit
41 graf1 = {
42     'a': ['d', 'e'],
43     'b': ['d', 'e'],
44     'c': ['f'],
45     'd': ['a', 'b'],
46     'e': ['a', 'b'],
47     'f': ['c']
48 }
49
50 bipartit, v1, v2 = este_bipartit(graf1)
51 print(f"Graf1 este bipartit? {bipartit}")
52 if bipartit:
53     print(f"V1 = {v1}")

```

```

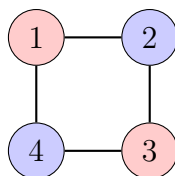
53     print(f"V2 = {v2}")
54
55 # Test 2: Triunghi (NU e bipartit)
56 graf2 = {'a': ['b', 'c'], 'b': ['a', 'c'], 'c': ['a', 'b']}
57 bipartit, _, _ = este_bipartit(graf2)
58 print(f"\nGraf2 (triunghi) este bipartit? {bipartit}")

```

Teoremă 3.1. *Un graf este bipartit dacă și numai dacă nu conține cicluri de lungime impară.*

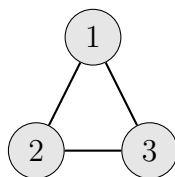
Exemplu 3.10. *Ilustrarea teoremei:*

Graf 1: *Ciclu C_4 (bipartit)*



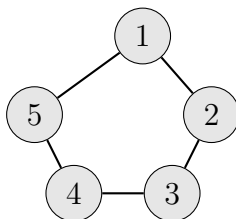
Lungime ciclu: 4 (par). Bipartit: $V_1 = \{1, 3\}$, $V_2 = \{2, 4\}$ ✓

Graf 2: *Ciclu C_3 (NU bipartit)*



Lungime ciclu: 3 (impar). NU poate fi colorat cu 2 culori. NU e bipartit. ✓

Graf 3: *Ciclu C_5 (NU bipartit)*



Lungime ciclu: 5 (impar). NU e bipartit. ✓

4 Drumuri și Conexitate

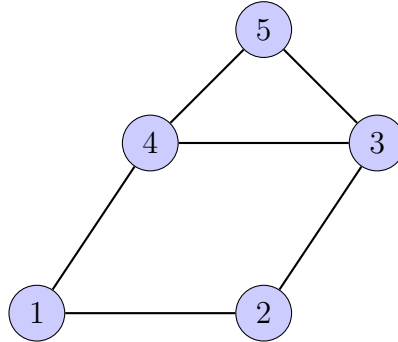
4.1 Drumuri în grafuri

Definiție 4.1. Fie $G = (V, E)$ un graf neorientat.

- Un **drum** (sau **cale**) în G este o succesiune de vârfuri $v_0, v_1, \dots, v_k$ astfel încât $\{v_i, v_{i+1}\} \in E$ pentru orice $i \in \{0, 1, \dots, k-1\}$;
- **Lungimea** unui drum este numărul de muchii din drum (în cazul de mai sus, k);

- Un drum este **simplic** dacă toate vârfurile sunt distincte (nu repetăm vârfuri);
- Un **ciclu** este un drum $v_0, v_1, \dots, v_k$ cu $k \geq 3$, $v_0 = v_k$ și $v_0, v_1, \dots, v_{k-1}$ distincte.

Exemplu 4.1. În graful de mai jos:



Drumuri:

- $1 \rightarrow 2 \rightarrow 3$ este un drum de lungime 2
- $1 \rightarrow 4 \rightarrow 3 \rightarrow 2$ este un drum de lungime 3
- $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4$ este un drum de lungime 4

Cicluri:

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ este un ciclu de lungime 4
- $3 \rightarrow 4 \rightarrow 5 \rightarrow 3$ este un ciclu de lungime 3 (triunghi)

Exemplu 4.2. Găsirea unui drum între două vârfuri folosind BFS:

```

1 def gaseste_drum_BFS(graf, start, dest):
2     """
3     Gaseste un drum de la start la dest folosind BFS.
4     Returneaza drumul sau None daca nu exista.
5     """
6     if start not in graf or dest not in graf:
7         return None
8
9     if start == dest:
10        return [start]
11
12    # BFS
13    coada = [[start]] # Fiecare element = drum partial
14    vizitate = {start}
15
16    while coada:
17        drum = coada.pop(0)
18        varf_curent = drum[-1]
19
20        for vecin in graf[varf_curent]:
21            if vecin == dest:
22                return drum + [vecin]
```

```

23         if vecin not in vizitate:
24             vizitate.add(vecin)
25             coada.append(drum + [vecin])
26
27
28     return None # Nu exista drum
29
30 # Test
31 graf = {
32     1: [2, 4],
33     2: [1, 3],
34     3: [2, 4, 5],
35     4: [1, 3, 5],
36     5: [3, 4]
37 }
38
39 drum = gaseste_drum_BFS(graf, 1, 5)
40 if drum:
41     print(f"Drum de la 1 la 5: {' -> '.join(map(str, drum))}")
42     print(f"Lungime drum: {len(drum) - 1}")
43 else:
44     print("Nu exista drum")
45
46 # Output: Drum de la 1 la 5: 1 -> 4 -> 5
47 #         Lungime drum: 2

```

Exemplu 4.3. Găsirea tuturor drumurilor simple între două vârfuri:

```

1 def gaseste_toate_drumurile(graf, start, dest, drum_curent=None,
2   toate_drumuri=None):
3     """
4     Gaseste toate drumurile simple de la start la dest.
5     """
6     if drum_curent is None:
7         drum_curent = [start]
8     if toate_drumuri is None:
9         toate_drumuri = []
10
11     if start == dest:
12         toate_drumuri.append(drum_curent[:])
13         return toate_drumuri
14
15     for vecin in graf.get(start, []):
16         if vecin not in drum_curent: # Evitam cicluri (drum
17             simplu)
18                 gaseste_toate_drumurile(graf, vecin, dest,
19                                         drum_curent + [vecin],
20                                         toate_drumuri)
21
22     return toate_drumuri
23
24 # Test

```

```

22 graf = {
23     1: [2, 4],
24     2: [1, 3],
25     3: [2, 4, 5],
26     4: [1, 3, 5],
27     5: [3, 4]
28 }
29
30 drumuri = gaseste_toate_drumurile(graf, 1, 5)
31 print(f"Toate drumurile simple de la 1 la 5:")
32 for i, drum in enumerate(drumuri, 1):
33     print(f"    Drum {i}: {' -> '.join(map(str, drum))} (lungime {
34         len(drum)-1})")
35
36 # Output:
37 # Drum 1: 1 -> 2 -> 3 -> 5 (lungime 3)
38 # Drum 2: 1 -> 2 -> 3 -> 4 -> 5 (lungime 4)
39 # Drum 3: 1 -> 4 -> 3 -> 5 (lungime 3)
40 # Drum 4: 1 -> 4 -> 5 (lungime 2)

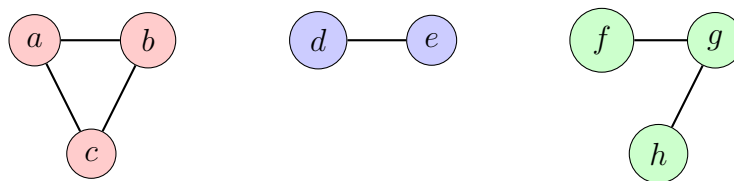
```

4.2 Conexitate

Definiție 4.2. • Două vârfuri u și v sunt **conectate** dacă există un drum de la u la v ;

- Un graf G este **conex** dacă oricare două vârfuri sunt conectate;
- O **componentă conexă** a unui graf este un subgraf conex maximal (nu poate fi extins cu alte vârfuri menținând conexitatea).

Exemplu 4.4. Graf cu 3 componente conexe:



Componente: $\{a, b, c\}$, $\{d, e\}$, $\{f, g, h\}$

Exemplu 4.5. Graf conex vs. graf neconex:



Exemplu 4.6. Verificare dacă un graf este conex:

```

1 def este_conex(graf):
2     """
3     Verifica daca un graf este conex.

```



```

4      Un graf e conex daca dintr-un varf putem ajunge la toate
      celelalte.
5      """
6      if not graf:
7          return True
8
9      # Pornim BFS din primul varf
10     start = list(graf.keys())[0]
11     vizitate = set()
12     coada = [start]
13     vizitate.add(start)
14
15     while coada:
16         varf = coada.pop(0)
17         for vecin in graf[varf]:
18             if vecin not in vizitate:
19                 vizitate.add(vecin)
20                 coada.append(vecin)
21
22     # Graful e conex daca am vizitat toate varfurile
23     return len(vizitate) == len(graf)
24
25 # Test 1: Graf conex
26 graf1 = {
27     'a': ['b', 'c'],
28     'b': ['a', 'c'],
29     'c': ['a', 'b', 'd'],
30     'd': ['c']
31 }
32 print(f"Graf1 este conex? {este_conex(graf1)}") # True
33
34 # Test 2: Graf neconex
35 graf2 = {
36     'a': ['b'],
37     'b': ['a'],
38     'c': ['d'],
39     'd': ['c']
40 }
41 print(f"Graf2 este conex? {este_conex(graf2)}") # False

```

Exemplu 4.7. Găsirea componentelor conexe:

```

1  def gaseste_componente_conexe(graf):
2      """
3      Returneaza lista de componente conexe.
4      Fiecare componenta = lista de varfuri.
5      """
6      vizitate = set()
7      componente = []
8
9      def BFS(start):
10         """BFS pentru a gasi o componenta conexa"""

```

```

11     componenta = []
12     coada = [start]
13     vizitate.add(start)
14
15     while coada:
16         varf = coada.pop(0)
17         componenta.append(varf)
18
19         for vecin in graf[varf]:
20             if vecin not in vizitate:
21                 vizitate.add(vecin)
22                 coada.append(vecin)
23
24     return componenta
25
26 # Gasim toate componentele
27 for varf in graf:
28     if varf not in vizitate:
29         componenta = BFS(varf)
30         componente.append(sorted(componenta))
31
32 return componente
33
34 # Test
35 graf = {
36     'a': ['b', 'c'],
37     'b': ['a', 'c'],
38     'c': ['a', 'b'],
39     'd': ['e'],
40     'e': ['d'],
41     'f': []
42 }
43
44 componente = gaseste_componente_conexe(graf)
45 print(f"Numar componente conexe: {len(componente)}")
46 for i, comp in enumerate(componente, 1):
47     print(f"Componenta {i}: {comp}")
48
49 # Output:
50 # Numar componente conexe: 3
51 # Componenta 1: ['a', 'b', 'c']
52 # Componenta 2: ['d', 'e']
53 # Componenta 3: ['f']

```

Definiție 4.3. *Distanța* între două vârfuri u și v , notată $d(u, v)$, este lungimea celui mai scurt drum între u și v (dacă există). Dacă nu există drum între u și v , atunci $d(u, v) = \infty$.

Exemplu 4.8. Calcularea distanței minime între două vârfuri (BFS):

```

1 def distanta_minima(graf, start, dest):
2     """

```

```

3      Calculeaza distanta minima (cel mai scurt drum) intre start
      si dest.
4      Returneaza lungimea drumului sau None daca nu exista drum.
5      """
6      if start == dest:
7          return 0
8
9      if start not in graf or dest not in graf:
10         return None
11
12     # BFS cu distante
13     coada = [(start, 0)] # (varf, distanta)
14     vizitate = {start}
15
16     while coada:
17         varf, dist = coada.pop(0)
18
19         for vecin in graf[varf]:
20             if vecin == dest:
21                 return dist + 1
22
23             if vecin not in vizitate:
24                 vizitate.add(vecin)
25                 coada.append((vecin, dist + 1))
26
27     return None # Nu exista drum
28
29 # Test
30 graf = {
31     1: [2, 3],
32     2: [1, 4],
33     3: [1, 4],
34     4: [2, 3, 5],
35     5: [4]
36 }
37
38 print(f"d(1, 5) = {distanta_minima(graf, 1, 5)}")
39 print(f"d(1, 4) = {distanta_minima(graf, 1, 4)}")
40 print(f"d(1, 1) = {distanta_minima(graf, 1, 1)}")
41
42 # Output:
43 # d(1, 5) = 3
44 # d(1, 4) = 2
45 # d(1, 1) = 0

```

Definiție 4.4. *Diametrul* unui graf conex G este:

$$\text{diam}(G) = \max_{u,v \in V} d(u, v)$$

adică cea mai mare distanță între oricare două vârfuri.

Exemplu 4.9. *Calcularea diametrului unui graf:*

```

1 def calculeaza_diametru(graf):
2     """
3     Calculeaza diametrul unui graf conex.
4     Diametru = max(distante intre toate perechile de varfuri).
5     """
6     if not este_conex(graf):
7         return None # Diametrul e definit doar pentru grafuri
8                     # conexe
9
10    diametru = 0
11    varfuri = list(graf.keys())
12
13    for i in range(len(varfuri)):
14        for j in range(i+1, len(varfuri)):
15            dist = distanta_minima(graf, varfuri[i], varfuri[j])
16            if dist is not None and dist > diametru:
17                diametru = dist
18
19    return diametru
20
21 # Test: Lant cu 5 varfuri
22 graf_lant = {
23     1: [2],
24     2: [1, 3],
25     3: [2, 4],
26     4: [3, 5],
27     5: [4]
28 }
29
30 print(f"Diametru lant: {calculeaza_diametru(graf_lant)}")
31 # Output: 4 (de la 1 la 5)
32
33 # Test: Graf complet K_4
34 k4 = {i: [j for j in range(1, 5) if j != i] for i in range(1, 5)}
35 print(f"Diametru K_4: {calculeaza_diametru(k4)}")
36 # Output: 1 (orice doua varfuri sunt direct conectate)

```

5 Exerciții propuse

1. Determinați numărul de muchii al unui graf n -regulat cu p vârfuri.
Indicație: Folosiți Lema strângerii de mână.
2. Demonstrați că în orice grup de 6 persoane, fie există 3 persoane care se cunosc reciproc, fie există 3 persoane care nu se cunosc reciproc.
3. Fie G un graf cu n vârfuri. Demonstrați că dacă $\deg(v) \geq \frac{n}{2}$ pentru orice vârf v , atunci G este conex.
4. Găsiți toate grafurile neizomorfe cu 4 vârfuri.

5. Scrieți un program Python care determină dacă un graf dat conține un triunghi (ciclu de lungime 3).
6. Fie T un arbore cu n vârfuri. Demonstrați că T are cel puțin 2 frunze (vârfuri de grad 1).
7. Implementați în Python un algoritm pentru a verifica dacă două grafuri date sunt izomorfe.
8. Determinați numărul de grafuri complete cu n vârfuri distincte (nu neapărat etichetate distinct).
9. Scrieți un program care generează toate grafurile cu 4 vârfuri (neizomorfe).
10. Demonstrați că un graf bipartit cu un număr impar de vârfuri nu poate fi regulat cu grad impar.